

MACHINE LEARNING FOR NLP ASSIGNMENT 2

DEADLINE: NOVEMBER 21ST 2025

General Description:

This assignment consists of a theoretical part that is a bit larger than Assignment 1 and the next steps in system evaluation. You will find the recommended planning indicating what to do in the first week and what in the second week at the bottom of the assignment.

Note: for this assignment, you will use the conll2003.train.conll data for training and the conll2003.dev.conll data for evaluation. For Assignment 3, you will need to confirm your final conclusions on the held-out test set conll2003.test.conll. You may want to save your models or already run your evaluations on both sets. Please note though that you should base your decisions on e.g. hyperparameters on your dev set and the test set is there to confirm the findings.

1. Theoretical Component

Goal of the assignment: system design and feature description of tasks other than NERC

During the module, we will have two working sessions where we explore task design. We focus in particular on *feature representation*. We first look at sentiment analysis, a task that already works relatively well with basic features. In the second part, we look at coreference resolution, a task for which more advanced features are needed.

1.A SemEval Shared Task 2013: Sentiment Analysis in Twitter

1.A.1 Provide a task definition of Subtask B. The task definition should be formulated with the goal of building a system in mind (what is the input, what the desired output).

1.A.2 Provide an overview of useful features based on what is mentioned in the paper describing the task and your discussions in class. Describe what they are and why they may be useful for the task. An example illustrating what the feature could do can be helpful in your descriptions.

1.A.3 Describe how the features you proposed could be presented for the system and what this means for how the system can use them (what does the computer see and how does this compare to what we see as humans).

1.B Coreference Resolution.

Coreference resolution is the task of determining which expressions in a text refer to the same thing in the real world. In this exercise, we will focus on entity co-reference, i.e. which expressions refer to the same entity. Take for instance the text below:

The police officer₁ came home from work. Her₁ husband₂ was in the kitchen₄. "I₂ got home early and already started cooking. Did you₁ manage to bring some tomatoes₃?" He₂ said. She₁ handed him₂ the tomatoes₃. "Peter₅ will stop by later this evening." Her₁ husband₂ continued.

The text above contains the following coreference chains:

1. police officer - her - you - She - Her
2. husband - I - He - him - husband
3. tomatoes - tomatoes

It also contains two singletons, entities that occur once: *kitchen* and *Peter*. The full task of entity coreference recognition consists of identifying which expressions refer to an entity in a text and determining which refer to the same entity. In this exercise, we will focus on the second step. The task is thus to determine whether two expressions referring to an entity refer to the same entity. Thus: do *police officer* and *her* refer to the same entity? Do *police officer* and *husband* refer to the same entity? Do *her* and *husband* refer to the same entity? Etc.

1.B.1 Provide an overview of useful features inspired by the rule-based system of Lee et al. 2013 and based on discussions we have had in class. Describe what they are and why they may be useful for the task. An example illustrating what the feature could do can be helpful in your descriptions.

1.B.2 Describe how the features you proposed could be presented for the system and what this means for how the system can use them (what does the computer see and how does this compare to what we see as humans). **You can restrict your answer to two types of features from your answer in 1.B.1 that involve different challenges.**

What to submit:

You can provide your answer in the *theoretical component* of the template.

2. Practical Component: Extending your System

For our Named Entity experiments, you will extend your system so that it covers more machine learning algorithms and more features. Carry out a basic literature search for named entity

recognition. This search need not be extensive, but is meant to make sure you include features that are easy to obtain and known to work. You do not need to write a related work review, but do write down the references of papers that gave you ideas for features (you will need to add those references to your feature descriptions).

You will first extend the system to cover more ‘traditional’ features, represented as one-hot encoding (as part of your presubmission) and then investigate word embeddings. An example of how to use word embeddings as a feature is provided in the code on GitHub. The setup is specifically designed so that you can combine word embeddings with other features later on.

2.1 Introducing More Features

You will expand the features you use further, starting with one-hot features and then moving on to word embeddings. You need to carry out the following steps:

- Identify which features you would like to use and what values these features can have (taking into account that they will be presented as one hot representation).
- Add those features to your system. Tip: if you plan to include some more advanced features, it can be helpful to first create a “feature extraction” script that identifies feature values and outputs them in the CoNLL format. Adding these features to your system becomes quite straight-forward this way.
- Integrate the option of using word embeddings to your system. The script provided as part of assignment2/ includes a function that obtains the feature representations already. The (commented out) example assumes you are using the Google word embeddings, which can be obtained here (search for “GoogleNews-vectors-negative300.bin.gz”). You only need to do this for one model, e.g. for SVM. You can decide to also experiment with multiple models if you are curious on how it interacts with different ones, but this is not necessary.

2.2 Including Alternative Machine Learning Methods

The code provided for Assignment 2 in the course github provides a setup that supports running experiments for multiple systems. It holds here as well that you are allowed to use everything that is provided, but you can also restructure the code or reimplement parts as you see fit (this does not affect your grade, unless your code becomes significantly worse).

Please extend your code so that you can also train models using:

- Naive Bayes: scikit learn provides multiple variations. Try to select the best one based on their descriptions.
- SVM.

Train and evaluate your systems with standard settings.

2.3 Hyper-parameters tuning

Read the [scikit-learn.org](https://scikit-learn.org/stable/tutorial/grid/parameter_tuning.html) documentation on hyper-parameter tuning (see this link) Carry out hyper-parameter tuning for your SVM. You need not explore all possible hyper-parameters and can choose which form of search you apply (exhaustive, randomized, halving). You can also choose whether you want to do cross-fold or tune on your development data. You'll need to provide a brief motivation of your choices.

3 Update Your NERC Report

Describe the work you did these two weeks as part of your system report:

- Update descriptions from Assignment 1 where appropriate (e.g., preprocessing based on your new features).
- Provide an overview of the features (with motivation) and systems you are using. If you use features that you have used based on the literature you have read, add references to this literature to your description.
- Provide an overview of your results (update your results with those obtained in this assignment). Please put your main results in LaTeX tables. More detailed results and images can be placed in the appendix.

Recommended Planning At the end of Week 1 from Module 2 (Week 3 of the course), we recommend you submit:

1. Theoretical component: the sentiment analysis on Twitter component (1.A.1, 1.A.2 and 1.A.3) (0,5 - 1 page)
2. Code/practical:
 - (a) Complete all one-hot features
 - (b) Run Naive Bayes and SVM (with one of the settings)
 - (c) Read up on hyper-parameter tuning so that you know what to do
3. The NERC report:
 - (a) Overview of features and their motivation
 - (b) Partial update of results (those that you obtained)

To be finished for the final submission (by the Assignment deadline)

1. Theoretical component: coreference resolution (1.B.1, 1.B.2) (1-2 pages).
2. Code/practical:
 - (a) Use pretrained word embeddings for (at least) one of the models

- (b) Run hyper-parameter tuning
3. The NERC report:
- (a) Revise writing where necessary
 - (b) Update feature description including a description of the word embeddings
 - (c) Describe hyper-parameter tuning in the designated section (what you did and what came out)
 - (d) Update results

4 What to submit

Please submit the pdf generated from your template and your code (a submission page for each will be provided). The pdf should contain:

- The description of your theoretical assignment included in the section Theoretical Component.
- An update of the preprocessing section if applicable.
- The new features you included (including word embeddings) in Section 3.2 Features.
- The models you ran this week in Section 3.1 Models.
- An overview and description of the results of your models in Section 4.1 Evaluation
- Please add a subsection Hyperparameter Tuning in the Section Experiments and Results and add a description of your Hyperparameter tuning results there.